

GEE_MCP_Plugin — Complete Setup & Architecture Guide

Natural-language control of Google Earth Engine, end to end. This document covers what was built, how it was built, every command that was run, and how to reproduce the whole stack on a fresh machine without AI assistance.

Project repository: https://github.com/Madhulika07-prog/GEE_MCP_Plugin **Author:** Madhulika Smriti

Generated: 2026-05-29

Table of Contents

1. [What This Is](#)
 2. [Architecture at a Glance](#)
 3. [The Development Workflow](#)
 4. [Part 1: The MCP Server](#)
 5. [Part 2: The Chat-Map Web App](#)
 6. [Part 3: Vector Upload, Exports, Asset Browsing](#)
 7. [Part 4: Multi-Provider LLM Layer](#)
 8. [Running Everything \(Commands Reference\)](#)
 9. [Reproducing on a Fresh Machine — Without AI](#)
 10. [Models & API Keys Reference](#)
 11. [Project File Reference](#)
 12. [Troubleshooting](#)
 13. [Extending the System](#)
 14. [Appendix: Complete Command History](#)
-

1. What This Is

Two independent but related ways to drive Google Earth Engine in plain English:

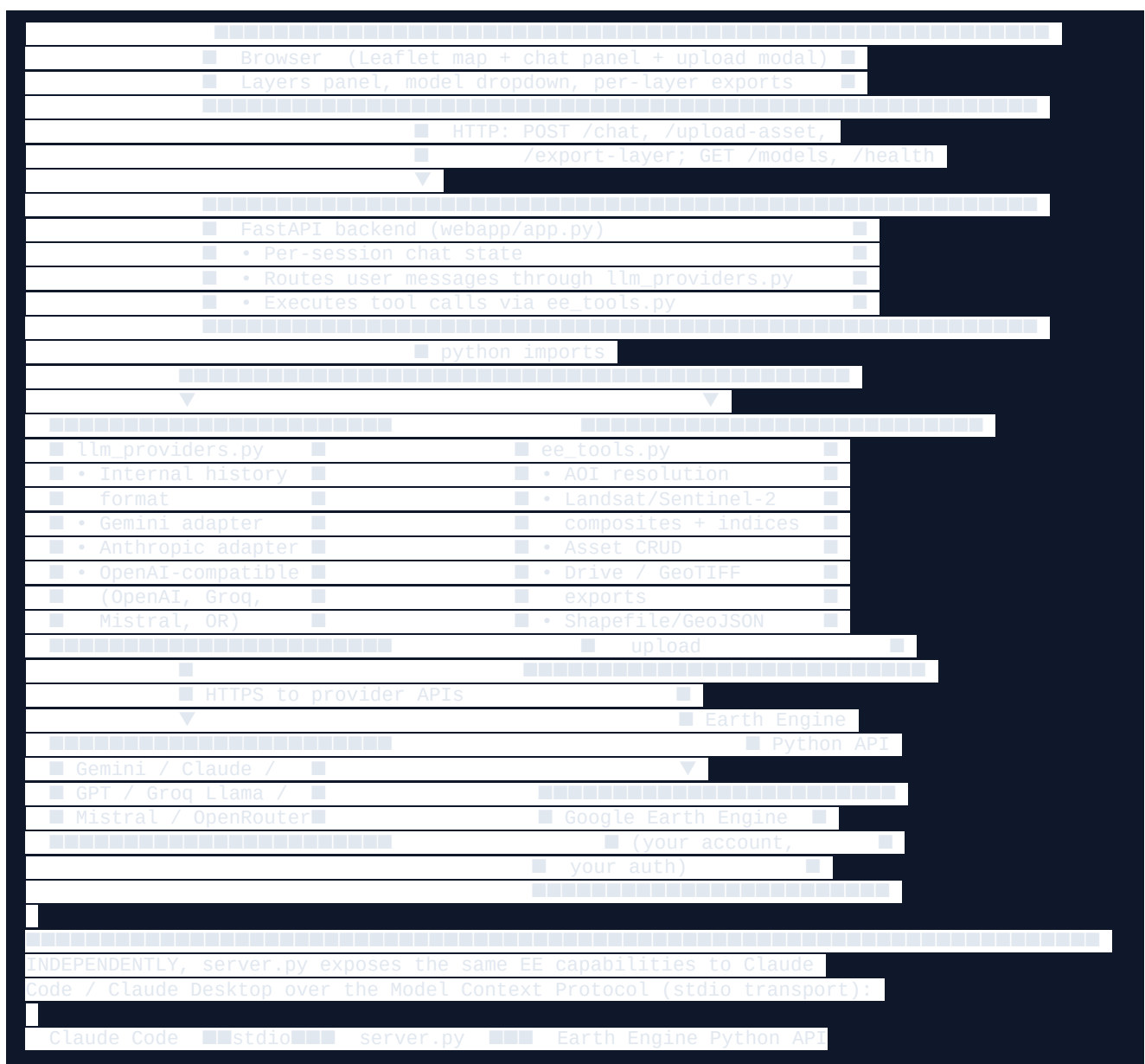
(a) An MCP server (`server.py`) that exposes Earth Engine as a set of tools to any client that speaks the Model Context Protocol — primarily Claude Code (the CLI / VSCode native extension) and Claude Desktop. When you ask Claude *"compute NDVI for Bangalore in May 2024"*, Claude calls our MCP tools instead of writing Python by hand; the server executes the call against your authenticated Earth Engine account and returns the result.

(b) A standalone chat-map web app (webapp/) — a single-page browser app with a chat panel on the left and a live Leaflet map on the right. Talk to the app, see EE layers appear on the map as live tiles. Built on top of the same EE tool functions, plus:

- A **multi-provider LLM layer** so you can switch between Gemini (free), Claude, GPT-4o, Llama-on-Groq (free), Mistral, and OpenRouter mid-conversation
- **Vector data upload** — drag in a GeoJSON or a zipped Shapefile and it becomes a usable AOI immediately, with automatic reprojection to WGS84
- **Per-layer exports** — Drive task or short-lived direct GeoTIFF download
- **Asset browsing** — list and visualize your own Earth Engine assets

The web app is single-user and runs on `localhost:8765`. It uses your own EE auth credential and your own LLM API key. No data flows through any third-party server we control.

2. Architecture at a Glance



The two surfaces share Earth Engine but not the LLM. When you talk to the web app, the LLM is whatever you pick in the dropdown. When you talk to Claude Code with the MCP enabled, the LLM is Claude itself — and the MCP gives it the EE tools.

3. The Development Workflow

This project was built collaboratively in a Claude Code session running inside the VS Code extension. Understanding that workflow helps you understand what's already on your machine vs what you need to install when reproducing elsewhere.

How the assistant accesses the terminal

Claude Code (the VSCode extension at code.visualstudio.com/extensions → install "Claude Code") gives an in-editor AI assistant tools to:

- **Read and write files** on the workstation
- **Run shell commands** via a sandboxed Bash tool (Git Bash on Windows, bash elsewhere)
- **Run PowerShell** for Windows-specific operations
- **Edit existing files** through a structured Edit tool that requires a prior Read
- **Search the codebase** via `globs` (filename patterns) and `grep` (content / regex)
- **Make HTTP requests** to URLs (used for fetching documentation)
- **Launch sub-agents** for parallel exploration

Every action runs *on the user's machine*. There is no remote build server. When the assistant says "running `python -m pip install x`", that pip command literally executes in the Windows terminal on the user's PC.

The user sees the commands and their output streamed into the chat. Many actions need explicit permission ("you're about to write to settings.json — allow?"). Risky actions (git force-push, `rm -rf`, etc.) require explicit confirmation in the user-facing chat.

How a request gets executed

A request like "add a chat-map web app" triggers, in order:

1. The assistant plans the work (often using a `TodoWrite` task list to track progress)
2. It asks clarifying questions where the answer materially changes the build
3. It calls the right tool — `write` to create a new file, `edit` to modify, `Bash` to run a shell command, etc.
4. It tests its work as it goes — `python -c "import x"` to check imports, `curl localhost:8765/health` to confirm an endpoint
5. It iterates if a step fails

This guide captures the result. The *Appendix: Complete Command History* at the end of this document lists every shell command that was run, in chronological order, so you can re-run the same sequence by hand.

4. Part 1: The MCP Server

4.1 What is MCP?

The Model Context Protocol (MCP) is an open standard from Anthropic for exposing tools, data, and prompts to LLM-powered clients. An MCP server is a small program that advertises a set of tools (each with a name, a description, and a JSON-schema for its arguments). A client like Claude Code spawns the server as a subprocess, communicates with it over stdio (newline-delimited JSON-RPC), and lets the LLM call any advertised tool.

For our purposes, MCP turns "Earth Engine" into something Claude can hold in its head as a finite list of capabilities — `search_image_collection`, `compute_index`, `export_image_to_drive`, etc. — instead of having to write Python from scratch each time.

4.2 Installing Python and the EE library

The host machine for this build was Windows 11 with Python 3.14 installed at `C:\Python314\python.exe`. Anything newer than Python 3.10 should work.

```
# Verify Python
python --version
# Python 3.14.0

# Install the EE Python API and the MCP SDK
python -m pip install --upgrade earthengine-api mcp python-dotenv
```

After this you'll see scripts installed at `%APPDATA%\Python\Python314\Scripts\` (Windows) or `~/local/bin/` (macOS/Linux). The two relevant scripts are `earthengine` (CLI for auth and asset management) and `mcp` (the SDK's command runner).

4.3 Authenticating to Earth Engine

Earth Engine requires a Google account and a registered Cloud project. Sign up at <https://signup.earthengine.google.com> if you haven't.

```
# Opens a browser for OAuth – sign in with the Google account that owns
# your EE project. The refresh token gets saved at:
#   Windows: %USERPROFILE%\config\earthengine\credentials
#   Mac/Linux: ~/.config/earthengine/credentials
& "$env:APPDATA\Python\Python314\Scripts\earthengine.exe" authenticate
```

The MCP server reads that credential the same way any EE Python script does.

4.4 The MCP server itself

`server.py` is built on the Python MCP SDK's `FastMCP` helper, which lets you declare each tool with a decorator. Auth initialization is lazy — the server doesn't connect to EE until the first tool call, so import-time errors don't block the client from starting.

Key parts:

```

# Load the .env that sits next to this file (EE_PROJECT, optional service-account key)
load_dotenv(Path(__file__).with_name(".env"))

EE_PROJECT = os.environ.get("EE_PROJECT")
EE_SERVICE_ACCOUNT_KEY = os.environ.get("EE_SERVICE_ACCOUNT_KEY")

mcp = FastMCP("earth-engine")

def _ensure_ee() -> None
    """Lazy EE.Initialize() on first tool call."""
    global _initialized
    if _initialized:
        return
    if EE_SERVICE_ACCOUNT_KEY:
        # Headless: use a service account JSON key from disk
        sa_email = json.load(open(EE_SERVICE_ACCOUNT_KEY))["client_email"]
        ee.initialize(ee.ServiceAccountCredentials(sa_email, EE_SERVICE_ACCOUNT_KEY),
                      project=EE_PROJECT)
    else:
        # User OAuth from `earthengine authenticate`
        ee.initialize(project=EE_PROJECT)
    _initialized = True

@mcp.tool()
def authenticate_status() -> dict
    """Report whether EE is reachable and which project/credential is in use."""
    _ensure_ee()
    _ = ee.Number(1).getInfo() # tiny round-trip
    return {"ok": True, "project": EE_PROJECT, ...}

```

The server registers **13 tools**:

Tool	What it does
<code>authenticate_status</code>	Sanity check — confirms EE is reachable
<code>search_image_collection</code>	Find scenes in a date range over an AOI
<code>image_metadata</code>	Bands, date, footprint, properties of a single image
<code>compute_index</code>	NDVI / NDWI / NBR / EVI over an AOI/date range
<code>get_thumbnail_url</code>	Quick PNG preview without a full export
<code>export_image_to_drive</code>	Start an Earth Engine → Drive export task
<code>export_image_to_gcs</code>	Start an EE → Cloud Storage export task
<code>get_download_url</code>	Short-lived direct GeoTIFF link (~32 MB cap)
<code>list_tasks</code>	Recent export tasks
<code>list_assets</code>	Browse your EE asset folders
<code>describe_asset</code>	Type, owner, properties of one asset
<code>delete_asset</code>	Gated by <code>confirm=True</code> for safety

```
run_ee_code
```

Generic escape hatch — execute arbitrary Python with ee available

4.5 Configuring Claude Code to use the MCP

The Claude Code VSCode extension reads MCP server configuration from `~/.claude.json` (not `~/.claude/settings.json` — that's a separate file for non-MCP settings).

Add this to your `~/.claude.json` (typically `C:\Users\<you>\.claude.json` on Windows):

```
{
  "mcpServers": {
    "earth-engine": {
      "type": "stdio",
      "command": "C:\\Python314\\python.exe",
      "args": ["C:\\Users\\<you>\\gee-mcp\\server.py"]
    }
  }
}
```

Then reload the VS Code window (Command Palette → "Developer: Reload Window"). The MCP server gets started by Claude Code automatically; you don't need to launch it yourself.

■ ■ **Common pitfall:** putting `mcpServers` in `~/.claude/settings.json` instead of `~/.claude.json` will silently fail — Claude Code looks for MCP servers in the second file, not the first.

4.6 Configuring Claude Desktop

Claude Desktop reads MCP from `claude_desktop_config.json`. On Windows this lives at `%APPDATA%\Claude\claude_desktop_config.json`. The schema is identical:

```
{
  "mcpServers": {
    "earth-engine": {
      "command": "C:\\Python314\\python.exe",
      "args": ["C:\\Users\\<you>\\gee-mcp\\server.py"]
    }
  }
}
```

A ready-to-copy snippet is provided as `claude_desktop_config.snippet.json` in the repository root.

4.7 Verifying the MCP works

Inside Claude Code (or Claude Desktop) after reload, ask:

"Use the earth-engine MCP and check `authenticate_status`."

You should see a result like:

```
{
  "ok": true,
  "project": "ee-yourproject",
  "auth_mode": "user_oauth"
}
```

If you see `"ok": false`, the most common causes are:

1. `EE_PROJECT` is missing from `gee-mcp/.env`
2. You haven't run `earthengine authenticate`
3. The Google account you authenticated with doesn't own the project named in `EE_PROJECT`

5. Part 2: The Chat-Map Web App

The web app builds on the same `ee_tools.py` logic the MCP uses, but adds a browser UI with a live map and a chat panel that you can drive with any LLM (not just Claude).

5.1 Stack choice

Backend: FastAPI + Uvicorn. Python is forced anyway by the EE library, and FastAPI's automatic OpenAPI docs and type-driven routing made this straightforward.

Frontend: Vanilla HTML/CSS/JS + Leaflet for the map. No React, no build step. This keeps the project a one-step install: `pip install -r requirements.txt` is enough.

Tile rendering: Earth Engine has a built-in `getMapId()` that returns a `tile_fetcher.url_format` string in standard `{z}/{x}/{y}` XYZ format. Leaflet's `L.tileLayer(template)` consumes that directly — no proxy, no static-image generation. Tiles load on demand from Google's CDN.

5.2 Backend: `webapp/app.py`

The backend has six routes:

Method	Route	Purpose
GET	<code>/</code>	Serve the single-page HTML
GET	<code>/health</code>	Report EE status + which provider keys are set
GET	<code>/models</code>	Return the model catalog with availability annotations
POST	<code>/chat</code>	The main chat endpoint — runs the LLM tool-use loop
POST	<code>/upload-asset</code>	Accept GeoJSON or zipped Shapefile → register + export to EE asset
POST	<code>/export-layer</code>	Re-build a layer's image and start a Drive export or return a download URL

POST	/reset	Clear a session's chat history
------	--------	--------------------------------

The `/chat` route is the heart of the app. Pseudocode:

```
@app.post("/chat")
async def chat(req: ChatRequest):
    session = SESSIONS.setdefault(req.session_id, {"history": [], "model_id": DEFAULT_MODEL})
    if req.model_id:
        session["model_id"] = req.model_id # allow mid-conversation switching
    text, ops = await run_in_threadpool(
        llm_providers.run_chat,
        model_id=session["model_id"],
        history=session["history"],
        user_message=req.message,
        tools=TOOLS,
        system=build_system_prompt(),
        execute_tool=execute_tool,
    )
    return {"session_id": req.session_id, "text": text, "operations": ops, "model_id": session["model_id"]}
```

The interesting bits:

- `history` is provider-agnostic — a list of `{role, blocks}` turns where each block is one of `text` / `tool_use` / `tool_result`. The provider adapter converts this to its native format on each call. This means **you can switch models mid-conversation** and the new model picks up where the old one left off.
- `build_system_prompt()` rebuilds the system prompt each turn, appending the names of any AOIs the user has uploaded via the modal. That's how the LLM learns "*Testing*" refers to a user-uploaded polygon.
- `execute_tool(name, args)` is a thin dispatcher: it maps the tool name to a function in `ee_tools.py`, calls it, returns both a model-visible result and an optional frontend operation that the browser will apply to the Leaflet map.

5.3 EE Tools: `webapp/ee_tools.py`

The functions used by the web app are similar in spirit to the MCP server's tools but tailored to produce *layers* (with tile URLs) rather than just stats.

- `add_ee_layer(source, visualize, aoi, year, months)` — composes a Landsat-or-Sentinel-2 median image, applies a visualization (RGB / NDVI / NDBI / LST), calls `getMapId()`, returns a tile URL + bounds.
- `add_admin_outline(aoi)` — resolves an admin polygon (FAO/GAUL → ADM2 → ADM1 → ADM0) and returns the geometry as GeoJSON for vector rendering.
- `compute_zonal_stats(...)` — mean/min/max/stdDev over the AOI.
- `list_my_assets(parent)` — `ee.data.listAssets({"parent": ...})`.
- `add_asset_layer(asset_id, ...)` — `ee.data.getAsset` to detect type, then load as Image or FeatureCollection.
- `export_layer_to_drive(...)` and `get_geotiff_url(...)` — re-build the image, then either start a `Export.image.toDrive` task or call `getDownloadURL()`.
- `list_export_tasks(limit)` — `ee.batch.Task.list()`.
- `upload_geojson_as_asset(geojson, asset_name, description)` — accepts a GeoJSON dict, constructs an `ee.FeatureCollection`, calls `Export.table.toAsset`. **No Google Cloud Storage bucket needed.**

- `shapefile_zip_to_geojson(zip_bytes)` — extracts the `.shp` / `.shx` / `.dbf` / `.prj` from a zip with `pyshp`, reads features, reprojects to WGS84 if needed via `pyproj`.

`resolve_aoi(aoi)` is the smart polymorphic resolver. It accepts:

1. A full EE asset path (`projects/ee-foo/assets/my_layer`)
2. A name from the in-memory upload cache (populated by `/upload-asset`)
3. A GAUL admin name (Bangalore → Bangalore Urban via aliases → ADM2 → ADM1 → ADM0)
4. A GeoJSON dict (Geometry / Feature / FeatureCollection)
5. A `{"bbox": [w, s, e, n]}` dict

This is why a single string field in the tool schema is enough — `resolve_aoi` figures out what the user meant.

5.4 Frontend: `webapp/templates/index.html`, `static/app.js`, `static/style.css`

The page is a flex split:

- **Left (~440 px):** chat panel. Header with the model dropdown, upload button, reset button. Scrollable message list. Textarea + Send button.
- **Right (rest):** Leaflet map with CARTO Light basemap. Floating layers panel top-right.

`app.js` keeps an in-memory Map of layer name → `{leaflet: <L.Layer>, meta: {...}}`. When `/chat` returns `operations: [{op: "add_layer", layer: {...}]`, the frontend creates a `L.TileLayer` from the tile URL, adds it to the map, and stores it in the registry. The layers panel re-renders to show toggle checkboxes + `■` export + `×` remove for each entry.

The model dropdown is populated from `/models` on page load. The list is grouped by provider; unavailable models (no key) are rendered disabled with a tooltip saying which env var to set.

5.5 What "running" looks like

```
# From the project root:
cd C:\Users\steen\gee-mcp\webapp
python -m uvicorn app:app --host 127.0.0.1 --port 8765
```

Then open `http://127.0.0.1:8765` in your browser. The first request to `/chat` triggers lazy EE initialization; you'll see `GET /health` and `POST /chat` log lines in the terminal.

6. Part 3: Vector Upload, Exports, Asset Browsing

6.1 Upload — GeoJSON

The upload modal accepts a GeoJSON file (FeatureCollection, Feature, or bare Geometry). The backend:

1. Reads the file (UTF-8, 20 MB cap)
2. JSON-parses it
3. Sanity-checks it's an object with `"type"`

4. Caches the geometry in `ee_tools._UPLOADED_AOIS[name]` so it's usable as an AOI **immediately**
5. Calls `ee.batch.Export.table.toAsset(...)` to persist it to your EE assets

The Export task takes 1–3 minutes to ingest, but the in-memory cache makes the AOI usable in chat right away. The next time you say "compute NDVI for Testing", `resolve_aoi("Testing")` checks the cache first and finds the geometry without waiting on EE.

6.2 Upload — Shapefile (zipped)

Shapefiles are composite — `.shp` + `.shx` + `.dbf` + (optionally) `.prj`. Users zip the four together and upload the `.zip`. The backend uses `pyshp` to read the shapefile and `pyproj` to reproject to WGS84 if the `.prj` indicates a different CRS.

```
def shapefile_zip_to_geojson(zip_bytes: bytes) -> dict:
    z = zipfile.ZipFile(io.BytesIO(zip_bytes))
    base = next(n for n in z.namelist() if n.lower().endswith(".shp"))[:-4]

    sf = shapefile.Reader(
        shp=io.BytesIO(z.read(f"{base}.shp")),
        shx=io.BytesIO(z.read(f"{base}.shx")),
        dbf=io.BytesIO(z.read(f"{base}.dbf")),
    )

    features = []
    for sr in sf.shapeRecords():
        geom = sr.shape.__geo_interface__ # pyshp gives GeoJSON-compatible dict
        props = dict(zip(field_names, sr.record))
        features.append({"type": "Feature", "geometry": geom, "properties": props})

    # Reproject if the .prj isn't WGS84
    if prj_text and not src_crs.equals(CRS.from_epsg(4326)):
        transformer = Transformer.from_crs(src_crs, "EPSG:4326", always_xy=True)
        # recurse through coordinate arrays and transform each pair
        ...

    return {"type": "FeatureCollection", "features": features}
```

The user just sees: "Uploaded Shapefile with 47 features. You can now use Pune_test as an AOI."

6.3 Exports

Two flavors:

Drive export task — works for any size. Uses `ee.batch.Export.image.toDrive(...)` and returns a `task_id`. The user tracks progress at <https://code.earthengine.google.com/tasks>; the resulting GeoTIFF lands in their Google Drive once the task completes.

Direct GeoTIFF URL — uses `ee.Image.getDownloadURL(...)`. EE caps this at ~32 MB per call, so it's only useful for small AOIs (e.g. one city at 30 m resolution). The URL is short-lived (~24 h) but downloads start immediately.

The frontend exposes both via the  button on each raster layer. The export panel includes:

- Format dropdown (Drive / direct URL)
- Scale (m) — pixel resolution
- File name prefix
- Drive folder (only when Drive is selected)

The export action posts to `/export-layer` with the same meta (source, visualize, aoi_input, year, months) that built the layer. Backend re-runs the same composite, then calls the appropriate EE method.

6.4 Asset Browsing

`list_my_assets("")` calls `ee.data.listAssets({"parent": "projects/<EE_PROJECT>/assets"})`. The result enumerates everything in your project root. Each entry has an `id`, `type` (IMAGE / IMAGECOLLECTION / TABLE / FOLDER), and `updateTime`.

`add_asset_layer(asset_id)` detects the type and dispatches:

- **IMAGE** → loaded with `ee.Image(asset_id)`; visualized with sane defaults (RGB stretch if it has 3+ bands, single-band palette otherwise)
- **IMAGECOLLECTION** → mean composite, then same as IMAGE
- **TABLE** (FeatureCollection) → loaded with `ee.FeatureCollection(asset_id)`; returned as GeoJSON for vector rendering (capped at 5,000 features for client safety)

7. Part 4: Multi-Provider LLM Layer

The web app supports six LLM providers behind one abstraction. The internal message format is provider-agnostic; each adapter converts it to and from its native format.

7.1 Internal format

```
history = [
  {
    "role": "user", "blocks": [
      {
        "type": "text", "text": "Hello"
      }
    ]
  },
  {
    "role": "assistant", "blocks": [
      {
        "type": "text", "text": "Hi! What can I do?"
      }
    ]
  },
  {
    "type": "tool_use", "id": "call_abc", "name": "list_my_assets", "args": {"parent": ""}
  },
  {
    "role": "user", "blocks": [
      {
        "type": "tool_result", "tool_use_id": "call_abc",
        "content": '{"asset_count": 4, "assets": [...]}'
      }
    ]
  }
]
```

This lets you switch from Gemini to Claude mid-conversation — the new adapter rebuilds the prior history into its own provider format.

7.2 The provider adapters

Adapter	SDK	Wire format
Gemini	google-genai	<code>genai_types.Content(role, parts=[...])</code> with <code>Part.from_text()</code> / <code>Part.from_function_call()</code> / <code>Part.from_function_response()</code>

Anthropic Claude	anthropic	Blocks: {"type": "text", ...} / {"type": "tool_use", "id", "name", "input"} / {"type": "tool_result", "tool_use_id", "content"}
OpenAI-compatible	openai	Messages with role, content, optional tool_calls; role: "tool" messages for results. Same shape works for OpenAI, Groq, Mistral, OpenRouter — only the base_url and key change.

7.3 The catalog

```
MODEL_CATALOG = [
    ModelSpec("google", "gemini-2.5-flash", "Gemini 2.5 Flash", free=True, ...),
    ModelSpec("google", "gemini-2.5-pro", "Gemini 2.5 Pro", free=False, ...),
    ModelSpec("groq", "llama-3.3-70b-versatile", "Llama 3.3 70B (Groq)", free=True, ...),
    ModelSpec("mistral", "mistral-small-latest", "Mistral Small", free=True, ...),
    ModelSpec("openrouter", "meta-llama/llama-3.3-70b-instruct:free", ..., free=True),
    ModelSpec("anthropic", "claude-opus-4-8", "Claude Opus 4.8", free=False, ...),
    ModelSpec("openai", "gpt-4o", "GPT-4o", free=False, ...),
    # ... and a few more per provider
]

PROVIDER_CONFIG = {
    "google": {"env": ("GEMINI_API_KEY", "GOOGLE_API_KEY"), "base_url": None},
    "anthropic": {"env": ("ANTHROPIC_API_KEY",), "base_url": None},
    "openai": {"env": ("OPENAI_API_KEY",), "base_url": None},
    "groq": {"env": ("GROQ_API_KEY",), "base_url": "https://api.groq.com/openai/v1"},
    "mistral": {"env": ("MISTRAL_API_KEY",), "base_url": "https://api.mistral.ai/v1"},
    "openrouter": {"env": ("OPENROUTER_API_KEY",), "base_url": "https://openrouter.ai/api/v1"}
}
```

A model is "available" if its env var is set. The frontend grays out unavailable models in the dropdown with a tooltip showing which env var to set.

7.4 Adding a new provider

Edit `llm_providers.py`:

1. Add an entry to `PROVIDER_CONFIG` with the env var name and base URL (or `None` if it has its own SDK)
2. If it's OpenAI-compatible, you're done — `_openai_run` already handles it
3. If it has its own SDK, write a new `_yourprovider_run(...)` adapter following the Gemini / Anthropic patterns
4. Wire the new adapter into the `run_chat` dispatcher
5. Add the model(s) to `MODEL_CATALOG`

8. Running Everything (Commands Reference)

8.1 One-time setup commands

```
# Python deps (run once, or after pulling new requirements.txt)
python -m pip install --upgrade -r requirements.txt

# Earth Engine OAuth (run once per machine)
& "$env:APPDATA\Python\Python314\Scripts\earthengine.exe" authenticate

# Copy .env template and fill in your values
copy env.example .env
# Edit .env in Notepad and set EE_PROJECT + at least one LLM key
```

8.2 Day-to-day commands

```
# Start the web app
cd C:\Users\steen\gee-mcp\webapp
python -m uvicorn app:app --host 127.0.0.1 --port 8765

# Open the UI
start http://127.0.0.1:8765
```

The MCP server doesn't get launched by hand — Claude Code / Claude Desktop launches it as a subprocess whenever you start a conversation in those apps.

8.3 Stop the web app

In the terminal where uvicorn is running: **Ctrl+C**.

9. Reproducing on a Fresh Machine — Without AI

This is the path you follow on a colleague's PC, a new laptop, or after a clean OS reinstall. No AI required; every step is a literal command or click.

9.1 Prerequisites

Software	Where to get it	Version
Python 3.11+	https://python.org/downloads	3.11 or newer (3.14 works)
Git for Windows	https://git-scm.com/download/win	latest
A Google account with EE access	https://signup.earthengine.google.com	—
(Optional) VS Code + Claude Code extension	https://code.visualstudio.com + ext from Anthropic	—
(Optional) Claude Desktop	https://claude.ai/download	—

Install Python with the "Add Python to PATH" checkbox **on**. Confirm in a new terminal:

```
python --version
git --version
```

9.2 Clone and install

```
# Clone the repo
git clone https://github.com/Madhulika07-prog/GEE_MCP_Plugin.git gee-mcp
cd gee-mcp

# Install Python deps
python -m pip install --upgrade -r requirements.txt
```

This installs everything: earthengine-api, mcp, fastapi, uvicorn, jinja2, google-genai, anthropic, openai, pyshp, pyproj, python-dotenv, python-multipart, pydantic.

9.3 Authenticate to Earth Engine

```
# Find the earthengine script (path varies by Python version):
where.exe earthengine
# If it's not on PATH, look in:
# %APPDATA%\Python\Python<XYZ>\Scripts\earthengine.exe

# Run authentication. A browser opens, sign in with the Google account that
# owns your EE project. The token is saved automatically.
earthengine authenticate
# or, with the full path:
& "$env:APPDATA\Python\Python314\Scripts\earthengine.exe" authenticate
```

9.4 Configure .env

```
copy .env.example .env
notepad .env
```

Set, at minimum:

```
EE_PROJECT=ee-yourproject

# Free option (recommended for first run):
GEMINI_API_KEY=AIzaSy... # get at https://aistudio.google.com/apikey
```

Add any of the optional keys if you want more model choices:

```
GROQ_API_KEY=gsk... # https://console.groq.com/keys (free)
MISTRAL_API_KEY=... # https://console.mistral.ai (free tier)
OPENROUTER_API_KEY=sk-or-... # https://openrouter.ai/keys (some free models)
ANTHROPIC_API_KEY=sk-ant-... # https://console.anthropic.com (paid)
OPENAI_API_KEY=sk-... # https://platform.openai.com (paid)
```

9.5 Start the web app

```
cd webapp
python -m uvicorn app:app --host 127.0.0.1 --port 8765
```

Open <http://127.0.0.1:8765> and try "Show NDVI over Bangalore for May 2024".

9.6 (Optional) Wire the MCP server into Claude Code

Edit `%USERPROFILE%\claude.json` and add:

```
"mcpServers":
  "earth-engine":
    "type": "stdio",
    "command": "C:\\Python314\\python.exe"
    "args": ["C:\\path\\to\\gee-mcp\\server.py"]
```

(Replace `C:\\Python314\\python.exe` with the output of `where.exe python` and `C:\\path\\to\\gee-mcp` with your actual clone path. Note the double backslashes — they're JSON-escaped.)

Reload the VS Code window. Test by asking Claude Code: *"check earth-engine authenticate_status"*.

9.7 (Optional) Wire the MCP server into Claude Desktop

Edit `%APPDATA%\\Claude\\claude_desktop_config.json` with the same structure (the file `claude_desktop_config.snippet.json` in the repo is a ready-to-copy template — just substitute your actual paths).

Quit and restart Claude Desktop.

9.8 Verifying the install

```
# Confirm Python imports succeed
python -c "import ee, fastapi, anthropic, google.genai, openai, shapefile, pyproj; print('OK!')"

# Confirm EE auth works
python -c "import ee; ee.Initialize(project='ee-yourproject'); print(ee.Number(1).getInfo())"
```

Both should print without errors. If the EE call fails, re-run `earthengine authenticate` and double-check `EE_PROJECT` in `.env`.

10. Models & API Keys Reference

10.1 The full catalog

Provider	Model ID	Cost tier	Notes
Google	gemini-2.5-flash	Free	Default — best balance of quality / speed / cost
Google	gemini-2.5-flash-lite	Free	Fastest; lower quality

Google	gemini-2.5-pro	Paid	Highest quality on Google
Groq	llama-3.3-70b-versatile	Free	Strong tool use; fast
Groq	llama-3.1-8b-instant	Free	Sub-second responses
Mistral	mistral-small-latest	Free tier	OK for chat
Mistral	mistral-large-latest	Paid	Strong reasoning
OpenRouter	meta-llama/llama-3.3-70b-instruct	Free	Proxied; rate-limited
OpenRouter	google/gemini-2.0-flash-exp	Free	Experimental Gemini through OR
Anthropic	claude-opus-4-8	Paid	Highest reasoning quality
Anthropic	claude-sonnet-4-6	Paid	Balanced
Anthropic	claude-haiku-4-5	Paid	Cheapest Claude
OpenAI	gpt-4o	Paid	OpenAI flagship
OpenAI	gpt-4o-mini	Paid	Cheaper GPT

10.2 Environment variables

`.env` lives next to `server.py` and `webapp/`. The web app reads it on every start; the MCP server reads it on every Claude Code reload.

Variable	Required?	What it does
EE_PROJECT	Yes	Your Earth Engine Cloud project ID, e.g. ee-madhulikasmriti
EE_SERVICE_ACCOUNT_KEY	No	Path to a service-account JSON if you want headless auth
GEMINI_API_KEY	At least one LLM key	Google AI Studio key
GOOGLE_API_KEY	(alternative to above)	Same as GEMINI_API_KEY
ANTHROPIC_API_KEY	Optional	Unlocks Claude models in the dropdown
OPENAI_API_KEY	Optional	Unlocks GPT models
GROQ_API_KEY	Optional	Unlocks Llama-on-Groq

MISTRAL_API_KEY	Optional	Unlocks Mistral
OPENROUTER_API_KEY	Optional	Unlocks OpenRouter models

10.3 Where to get each key

Provider	URL	Notes
Google Gemini	https://aistudio.google.com/apikey	Sign in with any Google account. No card needed. Keys start AIzaSy....
Groq	https://console.groq.com/keys	Sign up, generate a key. Free tier. Keys start gsk_....
Mistral	https://console.mistral.ai	Sign up, navigate to API Keys. Free tier on small models.
OpenRouter	https://openrouter.ai/keys	Free credits at signup; some models marked :free. Keys start sk-or-....
Anthropic	https://console.anthropic.com	Card required. Keys start sk-ant-....
OpenAI	https://platform.openai.com/api-keys	Card required. Keys start sk-....

10.4 Adding more models via the user-supplied key path

If you want to add a model that's not in the catalog, edit `webapp/llm_providers.py` and append to `MODEL_CATALOG`:

```
ModelSpec("groq", "qwen-qwq-32b", "Qwen QwQ 32B (Groq)", free=True notes="Reasoning")
```

The list refreshes on the next page load (/models is hit on boot).

11. Project File Reference

```
gee-mcp/
├── README.md # Project README
├── requirements.txt # All Python deps
├── .env.example # Template for the secrets file
├── .gitignore # Excludes .env, __pycache__, credential JSONs
├── claude_desktop_config.snippet.json # Ready-to-paste MCP config for Claude Desktop
└── server.py # MCP server (Claude Code / Claude Desktop)
```

```

docs/
  bangalore-uhi.md # Worked example: UHI analysis writeup
  setup-guide.md # This document (source)
  setup-guide.html # Rendered HTML
  setup-guide.pdf # Rendered PDF
  build_docs.py # Markdown → HTML + PDF builder script
  figures/
    lst_2015.png # Map: LST Mar-May 2015
    lst_2024.png # Map: LST Mar-May 2024
    lst_delta_2024_minus_2015.png # Map: ΔLST
    ndvi_2024.png # Map: NDVI 2024
webapp/
  app.py # FastAPI backend
  ee_tools.py # EE layer + asset + export logic
  llm_providers.py # Multi-LLM adapter
  templates/
    index.html # Single-page app
  static/
    app.js # Frontend JS (Leaflet + chat)
    style.css # All styling

```

12. Troubleshooting

Symptom	Cause	Fix
Browser shows "This site can't be reached"	uvicorn not running	<code>cd webapp && python -m uvicorn app:app --host 127.0.0.1 --port 8765</code>
<code>/health</code> says <code>ee_ok: false</code>	EE auth missing or wrong project	Run earthengine authenticate; check <code>EE_PROJECT</code> matches a project you own
<code>/health</code> says no provider keys set	Missing LLM key in <code>.env</code>	Get a free Gemini key, add to <code>.env</code> , restart server
MCP tools don't appear in Claude Code	Wrong config file	Should be <code>~/.claude.json</code> (top-level mcpServers), not <code>~/.claude/settings.json</code>
MCP shows up but <code>authenticate_status</code> fails	<code>EE_PROJECT</code> missing or wrong	Add to <code>gee-mcp/.env</code> , then reload the VS Code window
Chat returns empty text on Gemini	Thinking mode consumed all output tokens	The code disables thinking on 2.5 Flash; verify you're on the latest <code>llm_providers.py</code>

"Asset not found" on upload	EE asset ingestion task is still queued	Wait for the task to show COMPLETED in https://code.earthengine.google.com/tasks ; the AOI cache in memory makes the geometry usable in chat in the meantime
Shapefile upload fails	Missing .shx or .dbf in zip	Zip must contain .shp + .shx + .dbf; .prj is recommended
Per-layer  button missing	Layer is an asset/outline, not from add_ee_layer	Re-create the equivalent layer via chat (add_ee_layer flow stores the source params)
internal server error on /	Starlette 0.36+ requires request as first arg to TemplateResponse	Already fixed; pull latest app.py

13. Extending the System

13.1 Add a new EE tool

In `webapp/ee_tools.py`, write a Python function with simple args (strings, ints, lists):

```
def compute_swir_burn_ratio(aoi, year, months=None):
    ensure_ee()
    geom, _ = resolve_aoi(aoi)
    months = months or [3, 4, 5]
    start, end = window_dates(year, months)
    coll = landsat_sr(start, end, geom)
    img = coll.map(lambda i: ee.Image(i).normalizedDifference(["SWIR1", "SWIR2"]).rename("SBR").median())
    return img.reduceRegion(...).getInfo()
```

In `webapp/app.py`, add a tool declaration to `TOOLS` and a dispatch case in `_execute_tool`:

```
{
  "name": "compute_swir_burn_ratio",
  "description": "Burn severity proxy from Landsat SWIR1 vs SWIR2.",
  "parameters": {
    "type": "object",
    "properties": {
      "aoi": {"type": "string"},
      "year": {"type": "integer"},
      "months": {"type": "array", "items": {"type": "integer"}}
    }
  },
  "required": ["aoi", "year"],
},
# In _execute_tool:
if name == "compute_swir_burn_ratio":
    result = ee_tools.compute_swir_burn_ratio(**args)
    return result, None
```

Restart the server and the new tool is live.

13.2 Add a new LLM provider

See §7.4 above. The OpenAI-compatible path (Grok / Mistral / OpenRouter) is the easiest — just add the entry to `PROVIDER_CONFIG` with the right base URL.

13.3 Add raster image upload to EE

This needs a Google Cloud Storage bucket. Steps:

1. Create a GCS bucket: `gsutil mb -p ee-yourproject gs://your-bucket-name`
2. Service-account JSON with Storage Object Admin on that bucket
3. New endpoint /upload-raster that:
4. Accepts a GeoTIFF
5. Uploads to GCS via `google-cloud-storage`
6. Triggers `ee.data.startIngestion({"id": "projects/.../assets/<name>", "tilesets": [{"sources": [{"primaryPath": "gs://..."}]})`
7. Returns the task ID

The architecture supports it; the work is mostly wiring credentials and writing the GCS upload.

13.4 Add KML support

Add `fastkml` to requirements. In `ee_tools.py`:

```
def kml_to_geojson(kml_text: str) -> dict:
    from fastkml import kml
    k = kml.KML()
    k.from_string(kml_text)
    features = [...] # walk the KML tree and emit GeoJSON Features
    return {"type": "FeatureCollection", "features": features}
```

Wire into /upload-asset with a third branch (`fname.endswith(".kml")`).

14. Appendix: Complete Command History

Reproducing this project from scratch is essentially this sequence of commands. The actual development session interleaved them with file edits, but the install / configure / run shape is captured here.

A. Install Python deps

```
python -m pip install --upgrade earthengine-api mcp python-dotenv
python -m pip install fastapi "uvicorn[standard]" jinja2 anthropic python-multipart
python -m pip install google-genai
python -m pip install openai
python -m pip install pyshp pyproj
python -m pip install markdown xhtml2pdf pygments # for building these docs
```

Or, equivalently, once you have `requirements.txt`:

```
python -m pip install --upgrade -r requirements.txt
```

B. Authenticate to Earth Engine

```
# The earthengine script lives in Python's Scripts dir:  
& "$env:APPDATA\Python\Python314\Scripts\earthengine.exe" authenticate
```

A browser opens; sign in; the credential is saved to `~/.config/earthengine/credentials`.

C. Create `.env`

```
copy .env.example .env  
notepad .env
```

Set `EE_PROJECT` and at least one LLM API key.

D. Wire MCP into Claude Code

```
# Edit ~/.claude.json – note it's at ~/.claude.json, NOT ~/.claude/settings.json  
import json, pathlib  
p = pathlib.Path(r"C:\Users\<you>\.claude.json")  
d = json.loads(p.read_text(encoding="utf-8"))  
d["mcpServers"] = {  
    "earth-engine": {  
        "type": "stdio"  
        "command": r"C:\Python314\python.exe"  
        "args": [r"C:\Users\<you>\gee-mcp\server.py"]  
    }  
}  
p.write_text(json.dumps(d, indent=2), encoding="utf-8")
```

Then reload the VS Code window.

E. Run the web app

```
cd C:\Users\<you>\gee-mcp\webapp  
python -m uvicorn app:app --host 127.0.0.1 --port 8765
```

Open `http://127.0.0.1:8765`.

F. Smoke-test the wire-up

```
curl http://127.0.0.1:8765/health  
curl http://127.0.0.1:8765/models  
# Quick chat round-trip:  
curl -X POST http://127.0.0.1:8765/chat ^  
-H "Content-Type: application/json" ^  
-d '{"message": "list my earth engine assets"}'
```

G. Git setup and push

```
cd C:\Users\<you>\gee-mcp
|
git init -b main
git config user.email "you@example.com"
git config user.name "Your Name"
|
git add -A
git status --short          # verify .env is NOT staged
|
git commit -m "Initial commit: GEE MCP server + chat-map web app"
|
git remote add origin https://github.com/<you>/GEE_MCP_Plugin.git
git push -u origin main
```

The first push triggers Git Credential Manager — a browser window opens for GitHub sign-in. After one click it caches the credential.

H. Build the documentation

```
cd docs
python build_docs.py          # produces setup-guide.html + setup-guide.pdf
```

End of guide. Open the corresponding `setup-guide.html` in a browser for hyperlinked navigation, or `setup-guide.pdf` for a printable copy.